

# Aplicaciones

## Tema 5 · Parte 3: procesamiento de imágenes

Jesús Chacón Sombría

Departamento de Arquitectura de Computadores y Automática
Universidad Complutense de Madrid

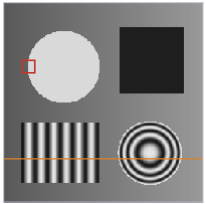
Grado en Ingeniería Electrónica de Comunicaciones
Curso 2026-2027

# Contenido

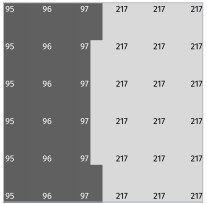

1. La imagen como señal 2D
2. Operaciones puntuales e histograma
3. Convolución 2D y filtrado espacial
4. DFT bidimensional
5. Filtrado en frecuencia

# Imagen digital

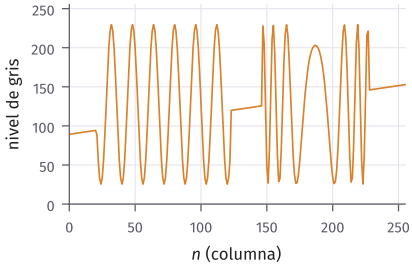
Imagen $f[m, n]$ de $256 \times 256$



Detalle de $16 \times 16$ píxeles




Perfil de la fila $m = 200$




- Una imagen es una señal de dos variables espaciales. Muestreada y cuantificada es una matriz $f[m, n]$ de $M \times N$ píxeles: $m$ es la fila (hacia abajo) y $n$ la columna.
- Escala de grises: un valor por píxel, normalmente entero de 8 bits (0 negro, 255 blanco) o real en $[0, 1]$.
- Color: tres canales (rojo, verde, azul), una matriz de $M \times N \times 3$.

# Muestreo, cuantificación y almacenamiento

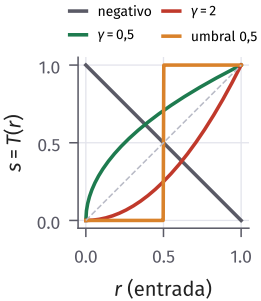
- Muestreo espacial: el sensor toma muestras con un paso $\Delta x$; la frecuencia espacial se mide en ciclos por píxel (o por milímetro). Si la escena tiene detalles más finos que $2\Delta x$ aparece aliasing: el efecto moiré de una camisa a rayas.
- Cuantificación: con $b$ bits hay $2^b$ niveles. Con pocos niveles aparecen contornos falsos en los degradados.
- Tamaño: $M \cdot N \cdot (\text{canales}) \cdot b/8$ bytes. Una imagen en color de 1920 × 1080 a 8 bits ocupa unos 6,2 MB sin comprimir.
- Formatos:
  - sin compresión (*raw*, BMP);
  - compresión sin pérdidas (PNG, TIFF): eliminan redundancia;
  - compresión con pérdidas (JPEG, JPEG 2000): descartan lo que menos se ve, con la DCT por bloques o con wavelets.
- Matlab: `imread`, `imshow`, `imwrite`. Python: `matplotlib.pyplot.imread` y `imshow`, o el paquete Pillow.

*Los ejemplos del tema usan una imagen sintética, `imagen_prueba`, para no depender de ficheros externos.*

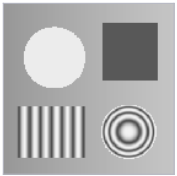
# Contenido


1. La imagen como señal 2D
2. Operaciones puntuales e histograma
3. Convolución 2D y filtrado espacial
4. DFT bidimensional
5. Filtrado en frecuencia

# Operaciones puntuales




γ = 0,5: aclara



γ = 2: oscurece



Umbral en 0,5



- Cada píxel se transforma con la misma función de su valor: $s = T(r)$. No intervienen los vecinos.
- Negativo $s = 1 - r$; corrección gamma $s = r^{\gamma}$ ($\gamma < 1$ aclara, $\gamma > 1$ oscurece); estiramiento lineal de $[r_1, r_2]$ a $[0, 1]$; umbralización, que da una imagen binaria.

## Histograma de una imagen

Con $L$ niveles $r = 0, \ldots, L-1$, $h[r]$ es el número de píxeles con nivel $r$. Normalizado, $p[r] = h[r]/(MN)$ es la proporción de píxeles de cada nivel, y su suma acumulada

$$F(r) = \sum_{i=0}^{r} p[i]$$

es la función de distribución de los niveles.

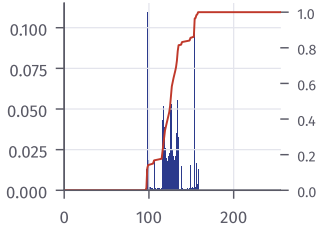
- Imagen oscura: histograma concentrado a la izquierda; clara, a la derecha; con poco contraste, estrecho.
- El histograma no dice dónde están los píxeles: dos imágenes muy distintas pueden tener el mismo. Es la misma idea que el histograma de muestras del tema 6.
- Permite elegir un umbral para separar objeto y fondo (por ejemplo, en el valle entre dos picos; método de Otsu).

# Ecualización del histograma

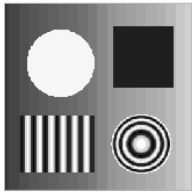
Poco contraste



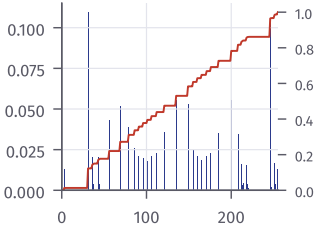
Histograma y $F(r)$




Ecualizada



Histograma ecualizado




- Se aplica la transformación $s = T(r) = \text{round}\big((L-1)\,F(r)\big)$, que es creciente.
- Si $r$ fuera continua, $s$ tendría distribución uniforme (tema 6: la distribución aplicada a su propia variable es uniforme). Con niveles discretos el histograma no queda plano, pero se reparte por todo el rango.

# En código: ecualización

**Python**

```python
import numpy as np
from imagen_prueba import imagen_prueba

f = imagen_prueba()
g = np.round(255*(0.35 + 0.3*f))
g = g.astype(int)            # contraste bajo
h = np.bincount(g.ravel(), minlength=256)
F = np.cumsum(h)/g.size      # distribución
T = np.round(255*F).astype(int)  # s = T(r)
ge = T[g]
print(g.min(), g.max())          # 97 158
print(ge.min(), ge.max())        # 3 255
print(np.std(g), np.std(ge))     # 16.7 73.3
```

**Matlab**

```matlab
f = imagen_prueba();
g = round(255*(0.35 + 0.3*f)); % 97..158
h = accumarray(g(:) + 1, 1, [256 1]);
F = cumsum(h)/numel(g);       % distribución
T = round(255*F);                % s = T(r)
ge = T(g + 1);
disp([min(g(:)) max(g(:))])      % 97 158
disp([min(ge(:)) max(ge(:))])    % 3 255
disp([std(g(:)) std(ge(:))])     % 16.7 73.3
% Con Image Processing Toolbox: histeq
```

*La desviación típica de los niveles, una medida del contraste, pasa de 17 a 73. Con la Image Processing Toolbox:* `imhist`, `histeq`, `imadjust`.

# Contenido


1. La imagen como señal 2D
2. Operaciones puntuales e histograma
3. Convolución 2D y filtrado espacial
4. DFT bidimensional
5. Filtrado en frecuencia

# Convolución bidimensional

## Convolución 2D

$$g[m,n] = (h * f)[m,n] = \sum_i \sum_j h[i,j]\, f[m-i, n-j]$$

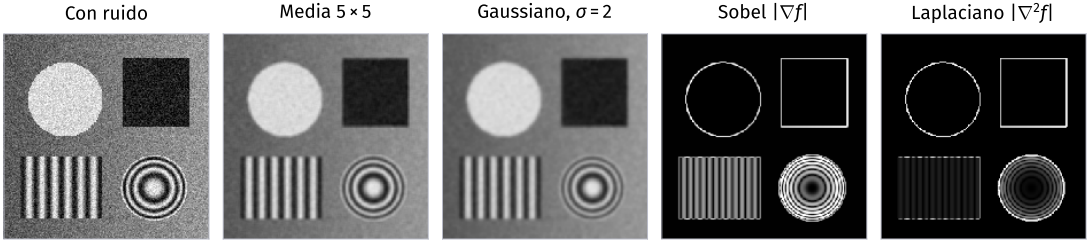
- $h$ es el núcleo (o máscara) del filtro, normalmente pequeño ($3 \times 3$, $5 \times 5$): un FIR 2D.
- Cada píxel de salida es una combinación de sus vecinos. Sistema lineal e invariante al desplazamiento.
- Imagen $M \times N$ y núcleo $K \times K$: la salida completa mide $(M + K - 1) \times (N + K - 1)$; con `'same'` se recorta al tamaño de la entrada.

- Bordes: fuera de la imagen hay que suponer algo: ceros, repetir el borde o reflejar.
- Separable si $h[i,j] = u[i]\, v[j]$: se filtran las filas con $v$ y las columnas con $u$. Cuesta $2K$ productos por píxel en vez de $K^2$.

$$\underbrace{\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}}_{\text{Sobel}} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

*`conv2` (Matlab), `convolve2d` (Python). `filter2` e `imfilter` calculan la correlación.*

# Núcleos habituales




- Paso bajo: media $\frac{1}{K^2}\mathbf{1}_{K\times K}$ o gaussiano $h[i,j] \propto e^{-(i^2+j^2)/(2\sigma^2)}$. Suavizan y reducen ruido; emborronan los bordes. El gaussiano no deja artefactos.
- Derivadas: Sobel aproxima $\partial f/\partial n$ y su traspuesto $\partial f/\partial m$; $|\nabla f| = \sqrt{g_x^2 + g_y^2}$ marca los bordes.
- Laplaciano $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$: segunda derivada, realza detalles y ruido.

# En código: bordes con Sobel



**Python**

```python
import numpy as np
from scipy.signal import convolve2d
from imagen_prueba import imagen_prueba

f = imagen_prueba()
sx = np.array([[1, 0, -1],
               [2, 0, -2],
               [1, 0, -1]])      # d/dn
media = np.ones((5, 5))/25
gx = convolve2d(f, sx, mode="same")
gy = convolve2d(f, sx.T, mode="same")
g = np.hypot(gx, gy)           # |gradiente|
fs = convolve2d(f, media, mode="same")
bordes = g > 0.5
print(bordes.mean())            # 0.18
print(f[80, 10:20])             # degradado
print(gx[80, 10:20])           # 8*0.25/256
```

**Matlab**

```matlab
f = imagen_prueba();
sx = [1 0 -1;
      2 0 -2;
      1 0 -1];                    % d/dn
media = ones(5)/25;
gx = conv2(f, sx, 'same');
gy = conv2(f, sx', 'same');
g = hypot(gx, gy);              % |gradiente|
fs = conv2(f, media, 'same');
bordes = g > 0.5;
disp(mean(bordes(:)))            % 0.18
disp(f(81, 11:20))               % degradado
disp(gx(81, 11:20))             % 8*0.25/256
```

*En el degradado $f$ crece* 0,25/256 *por columna y Sobel da 8 veces esa derivada.*

# Contenido


1. La imagen como señal 2D
2. Operaciones puntuales e histograma
3. Convolución 2D y filtrado espacial
4. DFT bidimensional
5. Filtrado en frecuencia

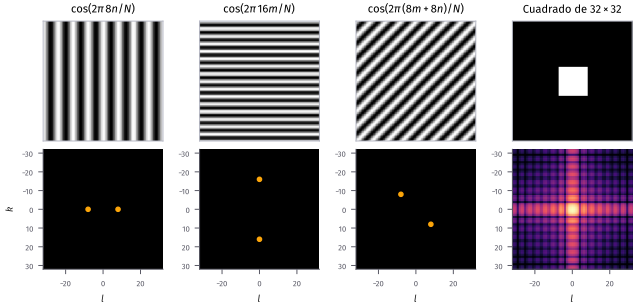
# Definición

## DFT 2D de una imagen $M \times N$

$$F[k,l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m,n]\, e^{-j2\pi\left(\frac{km}{M}+\frac{ln}{N}\right)}, \qquad f[m,n] = \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k,l]\, e^{j2\pi\left(\frac{km}{M}+\frac{ln}{N}\right)}$$

- Descompone la imagen en ondas planas $e^{j2\pi(km/M+ln/N)}$: franjas con $k$ ciclos en vertical y $l$ en horizontal. $F[0,0]$ es la suma de todos los píxeles.
- Separable: DFT 1D de cada fila y después de cada columna. Con la FFT cuesta $O(MN \log(MN))$. Matlab y Python: `fft2`, `ifft2`.
- Se dibuja $\log(1+|F|)$ con el origen en el centro (`fftshift`), porque $|F|$ abarca muchos órdenes de magnitud.

# Ejemplos



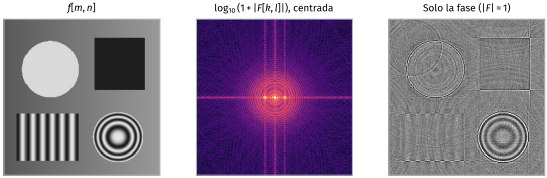

- Un coseno da dos puntos simétricos, en la dirección *perpendicular* a las franjas.
- Más franjas por píxel, puntos más lejos del centro.
- Un cuadrado da un sinc en cada dirección (tema 3), con lóbulos en forma de cruz.
- Bordes rectos de la imagen producen rayas perpendiculares en la DFT.

# Propiedades

- Periodicidad: $F[k+M,l] = F[k,l+N] = F[k,l]$. Por eso se puede centrar con `fftshift`.
- Simetría: si $f$ es real, $F[-k,-l] = F^*[k,l]$.
- Desplazamiento (circular):

$$f[m-m_0, n-n_0] \overset{F}{\longleftrightarrow} \mathrm{e}^{-j2\pi\left(\frac{km_0}{M}+\frac{ln_0}{N}\right)} F[k,l]:$$

  el módulo no cambia.
- Rotación: girar la imagen gira la DFT el mismo ángulo.
- Convolución: la convolución circular 2D es el producto de las DFT. Para la lineal se rellena con ceros hasta $(M+K-1)\times(N+K-1)$.
- Linealidad y Parseval: $\sum |f|^2 = \frac{1}{MN}\sum |F|^2$.




*La fase lleva la posición de los bordes: con solo la fase se reconocen los objetos.*

*Erratas antiguas: faltaban M y N en la propiedad de desplazamiento, y la de escalado era la de la transformada continua, que no vale para la DFT.*

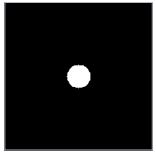
# Contenido


1. La imagen como señal 2D
2. Operaciones puntuales e histograma
3. Convolución 2D y filtrado espacial
4. DFT bidimensional
5. Filtrado en frecuencia

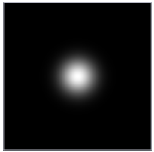
# Filtros en el dominio de la frecuencia

- $G[k,l] = H[k,l]\,F[k,l]$ y $g = \mathrm{IDFT}\{G\}$. Con $D[k,l]$ la distancia al origen (centrado):
- Ideal: $H = 1$ si $D \le D_0$. Su respuesta al impulso es un sinc 2D: anillos alrededor de los bordes.
- Gaussiano: $H = \mathrm{e}^{-D^2/(2D_0^2)}$. Equivale a convolucionar con un gaussiano de $\sigma = N/(2\pi D_0)$ píxeles.
- Paso alto: $1 - H_{\mathrm{PB}}$.

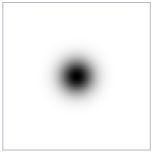
Paso bajo ideal, $D_0 = 20$



Paso bajo gaussiano, $D_0 = 20$



Paso alto gaussiano



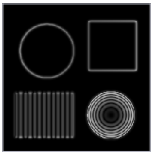
Anillos (fenómeno de Gibbs)



Suavizado sin anillos



Bordes (módulo)



# En código: paso bajo gaussiano con la DFT

**Python**

```python
import numpy as np
from imagen_prueba import imagen_prueba

f = imagen_prueba()
M, N = f.shape
F = np.fft.fft2(f)
k = np.fft.fftfreq(M)*M          # 0..-1
l = np.fft.fftfreq(N)*N
K, L = np.meshgrid(k, l, indexing="ij")
D2 = K**2 + L**2
D0 = 20
H = np.exp(-D2/(2*D0**2))        # gaussiano
g = np.real(np.fft.ifft2(H*F))
E = np.abs(F)**2
print(np.sum(E*H**2)/np.sum(E))  # 0.958
print(f.mean(), g.mean())        # H[0,0]=1
```

**Matlab**

```matlab
f = imagen_prueba();
[M, N] = size(f);
F = fft2(f);
k = [0:M/2-1, -M/2:-1];          % 0..-1
l = [0:N/2-1, -N/2:-1];
[L, K] = meshgrid(l, k);
D2 = K.^2 + L.^2;
D0 = 20;
H = exp(-D2/(2*D0^2));           % gaussiano
g = real(ifft2(H.*F));
E = abs(F).^2;
disp(sum(E(:).*H(:).^2)/sum(E(:)))
disp([mean(f(:)) mean(g(:))])    % H(1,1)=1
```

*$N = 256$, $D_0 = 20$: equivale a un gaussiano espacial de $\sigma \approx 2$ píxeles.*

# Más allá

- Restauración: deshacer un desenfoque conocido dividiendo por $H$ (con cuidado donde $H \approx 0$: filtro de Wiener).
- Compresión: JPEG aplica la DCT a bloques de $8 \times 8$ y cuantifica más las frecuencias altas; JPEG 2000 usa una DWT 2D.
- Segmentación y morfología: umbralizar, detectar bordes, rellenar y medir objetos.
- Visión por computador: las redes convolucionales encadenan muchas convoluciones 2D con núcleos aprendidos.

# Conceptos importantes

- Imagen: matriz $f[m, n]$ de $M \times N$ píxeles; uno o tres canales.
- Operaciones puntuales $s = T(r)$; histograma y ecualización con $T = (L - 1)F$.
- Convolución 2D con núcleos pequeños: suavizado, Sobel, laplaciano; separabilidad.
- DFT 2D separable; `fftshift` y $\log(1 + |F|)$.
- Periodicidad, simetría, desplazamiento, rotación, convolución.
- Filtrado en frecuencia: el ideal produce anillos; el gaussiano no.

Apuntes del Tema 5, sección 3 · Hoja de ejercicios, ejercicios 11–16.